



## 1A. Insert into an empty array

-No shift because there is no elements in the array making it  $O(1)$

| Test | nanoseconds |
|------|-------------|
| 1    | 228         |
| 2    | 262         |
| 3    | 261         |
| 4    | 321         |
| 5    | 243         |
| avg  | 263.2       |

## 1B. Insert when not full, no given index

### Insert at the back

inserting a number at the last index requires no shifting of any element in the array that's why it is fast.  $O(1)$

| Test | nanoseconds |
|------|-------------|
|------|-------------|

|            |     |
|------------|-----|
| <b>1</b>   | 284 |
| <b>2</b>   | 210 |
| <b>3</b>   | 166 |
| <b>4</b>   | 200 |
| <b>5</b>   | 196 |
| <b>avg</b> | 211 |

### Insert at the front

inserting a number at the first index requires shifting every element in the array to the left (worst case) that's why it is slow.  $O(n)$

|            |             |
|------------|-------------|
| Test       | nanoseconds |
| <b>1</b>   | 223         |
| <b>2</b>   | 232         |
| <b>3</b>   | 248         |
| <b>4</b>   | 238         |
| <b>5</b>   | 170         |
| <b>avg</b> | 222         |

## 1C. Insert at a given index

|             | <b>beginning</b>  | <b>middle</b> | <b>last</b> |
|-------------|-------------------|---------------|-------------|
| <b>Test</b> | <b>nanosecond</b> |               |             |
| <b>1</b>    | <b>493</b>        | <b>115</b>    | <b>159</b>  |

|            |            |     |     |
|------------|------------|-----|-----|
| <b>2</b>   | <b>266</b> | 233 | 140 |
| <b>3</b>   | <b>233</b> | 174 | 160 |
| <b>4</b>   | <b>258</b> | 147 | 205 |
| <b>5</b>   | <b>169</b> | 126 | 107 |
| <b>avg</b> | 283        | 159 | 154 |

## 1D. Insert when full size

### Insert with fixed buffer

No, you cannot insert a number into a max capacity array with a fixed buffer because we don't have any available space to store the number and a fixed buffer cannot grow.

### Insert into a max capacity array and resize the array

|             | <b>beginning</b>  | <b>last</b> | <b>Dynamic arr</b> |
|-------------|-------------------|-------------|--------------------|
| <b>Test</b> | <b>nanosecond</b> |             |                    |
| <b>1</b>    | <b>493</b>        | <b>159</b>  | 2288               |
| <b>2</b>    | <b>266</b>        | 140         | 1741               |
| <b>3</b>    | <b>233</b>        | 160         | 2545               |
| <b>4</b>    | <b>258</b>        | 205         | 2246               |
| <b>5</b>    | <b>169</b>        | 107         | 2519               |
| <b>avg</b>  | 283               | 154         | 1919               |

The reason why inserting into a dynamic array takes more time than the rest is because we have to resize the array and then copy every element of the old array into the new one and then insert the number in.

```

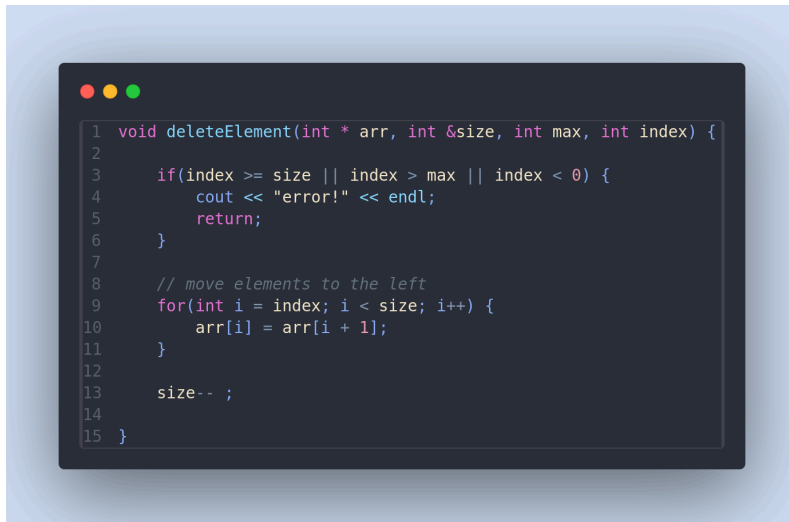
1
2 int main(){
3     const int MAX_CAP = 100000;
4     int n = 1;
5
6     int *arr = new int[n] {1};
7     int size = 1;
8
9     int pos = 0;
10
11     auto t0 = clk::now();
12
13     insertElement(arr, size, n, pos, 12);
14
15     auto t1 = clk::now();
16     cout << size << endl;
17     cout << chrono::duration_cast<chrono::nanoseconds>(t1 -
18 t0).count() << "\n";
19
20     for(int i = 0; i < size; i++){
21         cout << "value " << i << " : " << arr[i] << endl ;
22     }
23
24     delete[] arr;
25
26     return 0;
27 }

```

```

1 void expand(int *&arr, int &size, int &max) {
2     int * tempArr = new int[max*2];
3     max++ ;
4
5     for(int i = 0; i < size; i++){
6         tempArr[i] = arr[i];
7     }
8
9     delete[] arr;
10    arr = tempArr;
11 }
12
13 void insertElement(int *&arr, int &size, int &capacity, int
14 index, int num) {
15
16     if (index < 0 || index > size) {
17         cout << "Error: Invalid index " << index << endl;
18         return;
19     }
20
21     if (size == capacity) {
22         expand(arr, size, capacity);
23     }
24
25     // Shift elements to the right
26     for (int i = size; i > index; i--) {
27         arr[i] = arr[i - 1];
28     }
29
30     arr[index] = num;
31     size++;
32 }

```



## 2A. Remove from the middle

| Test       | nanoseconds |
|------------|-------------|
| <b>1</b>   | 228         |
| <b>2</b>   | 195         |
| <b>3</b>   | 256         |
| <b>4</b>   | 211         |
| <b>5</b>   | 149         |
| <b>avg</b> | 207         |

The reason why removing from the middle can take longer than removing the last element is because the algorithm has to be in a loop to shift all the elements after the middle element 1 place to the left , making the time complexity of this algorithm  $O(n)$ . This process may take longer than removing the last but still faster than removing the first element.

## 2B. Remove from the end

| Test     | nanoseconds |
|----------|-------------|
| <b>1</b> | 196         |
| <b>2</b> | 182         |

|            |     |
|------------|-----|
| <b>3</b>   | 198 |
| <b>4</b>   | 190 |
| <b>5</b>   | 191 |
| <b>avg</b> | 191 |

Remove from the end is the fast because when removing the last element the algorithm don't need to be in any loop to shift any of the element, making the time complexity for this is  $O(1)$

## 2C. Remove from the beginning

|            |             |
|------------|-------------|
| Test       | nanoseconds |
| <b>1</b>   | 229         |
| <b>2</b>   | 247         |
| <b>3</b>   | 252         |
| <b>4</b>   | 297         |
| <b>5</b>   | 289         |
| <b>avg</b> | 262         |

Remove element at the beginning of the array can take longer than the rest because the algorithm has to be in a loop to shift every element after the first element 1 place to the left  $n$  time, making the time complexity for this is  $O(n)$

## 2D. Edge sanity

|             | <b>2A</b>         | <b>2C</b> | <b>Edge</b> |
|-------------|-------------------|-----------|-------------|
| <b>Test</b> | <b>nanosecond</b> |           |             |
| <b>1</b>    | 228               | 229       | <b>177</b>  |
| <b>2</b>    | 195               | 247       | 178         |

|            |     |     |     |
|------------|-----|-----|-----|
| <b>3</b>   | 256 | 252 | 188 |
| <b>4</b>   | 211 | 297 | 250 |
| <b>5</b>   | 149 | 289 | 217 |
| <b>avg</b> | 207 | 262 | 202 |

Removing the only element in the array can be as fast as removing an element at the last index because both operations don't require any loop to shift any element since the remove element is already at the last index, making the time complexity for this operation is  $O(1)$

### 3. Single question

#### Access by index

\_ it does not require scanning the array. It can access the location of the element in the array with  $i$ . To prove this you can create 2 arrays with different sizes, one with 1000 elements and another one with 1 million elements. If you measure the average access time between the 2 arrays it will be almost identical. This proves that access's time by index does not grow with the amount of elements.

#### Linear search

\_ best case  $O(1)$  is when the target is the first element in the array. Worst case  $O(n)$  is when the target is the last element in the array or not even in the array, that means we have to scan  $n$  time in order to find the target and the scan time is based on the amount of  $n$ .